```
import copy

goal = [[1,2,3],
        [4,5,6],
        [7,8,0]]

moves = [(0,1),(1,0),(0,-1),(-1,0)]

def find_blank(board):
    for i in range(3):
        for j in range(3):
            if board[i][j] == 0:
                return i,j

def print_board(board):
    for row in board:
        print(row)
    print()

def dfs(state, visited, path, depth, limit):

    if state == goal:
        return path + [state]

    if depth >= limit:
        return None

    visited.add(str(state))

    x,y = find_blank(state)

    for dx,dy in moves:

        nx = x + dx
        ny = y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:

            new_state = [row[:] for row in state]
            new_state[x][y],new_state[nx][ny] = new_state[nx][ny],new_state[x][y]

            if str(new_state) not in visited:

                result = dfs(new_state, visited, path+[state], depth+1, limit)

                if result:
                    return result

    return None


initial = [[1,2,3],
           [4,0,6],
           [7,5,8]]

solution = dfs(initial, set(), [], 0, 10)
```

```
if solution:
    print("Solution Path:\n")
    for step in solution:
        print_board(step)
else:
    print("No Solution Found")
```

Output:

```
Solution Path:

[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

[1, 2, 3]
[4, 6, 0]
[7, 5, 8]

[1, 2, 3]
[4, 6, 8]
[7, 5, 0]

[1, 2, 3]
[4, 6, 8]
[7, 0, 5]

[1, 2, 3]
[4, 0, 8]
[7, 6, 5]

[1, 2, 3]
[4, 8, 0]
[7, 6, 5]

[1, 2, 3]
[4, 8, 5]
[7, 6, 0]

[1, 2, 3]
[4, 8, 5]
[7, 0, 6]

[1, 2, 3]
[4, 0, 5]
[7, 8, 6]

[1, 2, 3]
[4, 5, 0]
[7, 8, 6]

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```

=== Code Execution Successful ===